

DATA ANALYTICS WITH GENAI · SQL MINI PROJECT

Retail Store

SQL Solution

Documentation

A complete technical walkthrough of the retail_store database - schema, dataset, and all 42 business queries across six levels of the Data Query Language syllabus.

PREPARED BY	Awanish Verma
SOURCE FILE	Awanish Verma - Retail_Store_Complete_Solution_Explained.sql
ENGINE	MySQL 8.x
COVERAGE	42 / 42 questions

6

TABLES

42

QUERIES

6

LEVELS

2,131

ROWS LOADED

400

ORDERS

₹69.6L

REVENUE

01 - CONTEXT

Problem Statement

A retail business records every customer, product, order, line item, payment and review in an operational database - but the raw tables answer nothing on their own.

The store captures transactions correctly, but the people who need to act on that data cannot reach it. Marketing has no clean contact list, merchandising cannot see which categories perform, finance cannot break settlement down by method, and inventory cannot separate what sells from what sits.

- **Fragmentation.** One question - who paid for which order, and how - spans three tables.
- **Granularity mismatch.** Data is stored per line item; questions are asked per order, customer, category and day.
- **Absence is invisible.** "Never sold" and "never ordered" require reasoning about rows that are not there.

THE CORE PROBLEM

There is no documented, repeatable query layer translating these business questions into correct SQL against the `retail_store` schema, and no record of what each query returns.

02 - INTENT

Purpose of This Project

To build and document a complete, verified SQL answer set for the retail store, progressing from single-table retrieval to set operations.

The project has two audiences at once: the business, which needs answers, and the learner, who needs to understand how each answer was derived. Every query in this document therefore carries its business rationale, its mechanics and its verified result.

Objectives

- Create the six-table `retail_store` schema from the supplied DDL, with keys and constraints intact.
- Load the supplied dataset idempotently, so the script can be re-run without failure.
- Answer all 42 business questions across the six DQL levels using standard MySQL.
- Explain each query in plain language - what it selects, what it filters and why.
- Record the expected result of every query so output can be verified, not assumed.

- Document deviations forced by engine limits, such as the absence of `INTERSECT`.

Scope

IN SCOPE	DDL, DML loading, verification, and all Data Query Language questions from Levels 1 to 6 of the project brief.
OUT OF SCOPE	Stored procedures, triggers, views, indexing strategy, window functions and BI dashboarding - none are required by the brief.
ENGINE	MySQL 8.x. Syntax such as <code>AUTO_INCREMENT</code> and <code>INSERT IGNORE</code> is MySQL-specific.
DELIVERABLE	One consolidated script, <code>Retail_Store_Complete_Solution_Explained.sql</code> , plus this document.

03 - APPROACH

Proposed Solution

A single idempotent script that builds the database, loads the data, verifies the load, and then answers every question in ascending order of complexity.

The solution is structured as four sequential stages inside one file, so that a reviewer can execute it top to bottom on a clean server and arrive at exactly the results documented here.

Stage 1 - Schema construction

The script creates the database if it is absent, selects it, and creates all six tables with `CREATE TABLE IF NOT EXISTS`. Foreign key checks are disabled for the duration of table creation so that tables can be declared in any order without dependency errors, then re-enabled immediately afterwards.

```
CREATE DATABASE IF NOT EXISTS retail_store;
USE retail_store;

SET FOREIGN_KEY_CHECKS = 0;
-- ... six CREATE TABLE IF NOT EXISTS statements ...
SET FOREIGN_KEY_CHECKS = 1;
```

Stage 2 - Idempotent data load

The supplied dataset carries explicit primary key values. Running it twice against the same database would raise duplicate-key errors, so every statement uses `INSERT IGNORE`: rows that already exist are silently skipped rather than aborting the script.

Stage 3 - Verification

A single **UNION ALL** query reports the row count of all six tables at once, confirming the load before any analysis begins.

```
SELECT 'customers' AS table_name, COUNT(*) AS row_count FROM customers
UNION ALL
SELECT 'products', COUNT(*) FROM products
UNION ALL
SELECT 'orders', COUNT(*) FROM orders
UNION ALL
SELECT 'order_items', COUNT(*) FROM order_items
UNION ALL
SELECT 'payments', COUNT(*) FROM payments
UNION ALL
SELECT 'product_reviews', COUNT(*) FROM product_reviews;
```

table_name	row_count
customers	30
products	50
orders	400
order_items	1201
payments	400
product_reviews	50

Stage 4 - Layered query set

The 42 questions are answered in six levels, each introducing exactly one new class of technique and reusing everything before it. This ordering is deliberate: a reader who understands Level 3 aggregation can read a Level 5 correlated subquery as an aggregate evaluated per row.

LEVEL	FOCUS	QUESTIONS	CORE TECHNIQUE
L1	Basics	10	SELECT, WHERE, ORDER BY, DISTINCT
L2	Filtering & formatting	6	Aliases, arithmetic, CONCAT, DATE()
L3	Aggregations	10	COUNT, SUM, AVG, GROUP BY
L4	Joins	7	INNER, LEFT and RIGHT JOIN
L5	Subqueries	7	Scalar, correlated, EXISTS
L6	Set operations	2	UNION, join-based intersection

DESIGN PRINCIPLE

Every query is written to be readable before it is written to be clever. Keywords sit on their own lines, table aliases are single letters bound close to their table, and any calculated column is given an explicit alias so that the output is self-describing.

04 - FOUNDATION

Database Schema

Six tables, normalised so that each fact is stored once and referenced by key.

The design separates the parties (customers, products) from the events (orders, payments, reviews) and uses `order_items` as the bridge that resolves the many-to-many relationship between orders and products. `item_price` is stored on the line item rather than read from `products`, so that historical orders keep the price that was actually charged even after the catalog price changes.

customers

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
customer_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique ID for each customer
name	VARCHAR(100)	NOT NULL	Full name of the customer
email	VARCHAR(100)	UNIQUE, NOT NULL	Unique email address
phone	VARCHAR(15)	-	Optional phone number
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Account creation timestamp

products

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
product_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique product ID
name	VARCHAR(100)	NOT NULL	Name of the product
category	VARCHAR(50)	NOT NULL	Product category (e.g. Electronics)
price	DECIMAL(10,2)	NOT NULL	Product price
stock_quantity	INT	NOT NULL DEFAULT 0	Units available in stock

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
added_on	DATETIME	DEFAULT CURRENT_TIMESTAMP	Date the product was added

orders

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
order_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique order ID
customer_id	INT	FK → customers(customer_id)	Who placed the order
order_date	DATETIME	DEFAULT CURRENT_TIMESTAMP	When the order was placed
status	VARCHAR(20)	DEFAULT 'Pending'	Pending, Shipped, Delivered, Cancelled
total_amount	DECIMAL(10,2)	-	Calculated order total

order_items

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
order_item_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique ID for each line item
order_id	INT	FK → orders(order_id)	Order to which the item belongs
product_id	INT	FK → products(product_id)	Which product was bought
quantity	INT	NOT NULL CHECK (quantity > 0)	Quantity ordered
item_price	DECIMAL(10,2)	NOT NULL	Snapshot of product price at order time

payments

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
payment_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique ID for the payment
order_id	INT	FK → orders(order_id)	Which order was paid for
payment_date	DATETIME	DEFAULT CURRENT_TIMESTAMP	When payment was made
amount_paid	DECIMAL(10,2)	NOT NULL CHECK (amount_paid > 0)	Amount paid

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
method	VARCHAR(20)	NOT NULL	Credit Card, Debit Card, Net Banking, UPI

product_reviews

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
review_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique ID for the review
product_id	INT	FK → products(product_id)	Which product is reviewed
customer_id	INT	FK → customers(customer_id)	Who gave the review
rating	INT	NOT NULL CHECK (rating BETWEEN 1 AND 5)	Star rating
review_text	TEXT	-	Review message
review_date	DATETIME	DEFAULT CURRENT_TIMESTAMP	When the review was posted

Entity relationships

PARENT	CHILD	TYPE	FOREIGN KEY	MEANING
customers	orders	One-to-Many	orders.customer_id → customers.customer_id	A customer can place multiple orders
orders	order_items	One-to-Many	order_items.order_id → orders.order_id	An order can contain multiple items
products	order_items	One-to-Many	order_items.product_id → products.product_id	A product can appear in many different orders
orders	payments	One-to-Many	payments.order_id → orders.order_id	One or more payments per order
products	product_reviews	One-to-Many	product_reviews.product_id → products.product_id	A product can have many customer reviews
customers	product_reviews	One-to-Many	product_reviews.customer_id → customers.customer_id	A customer can write multiple reviews

05 - DATA

Dataset Profile

2,131 rows across six tables, spanning one year of retail trading.

TABLE	ROWS	CONTENTS
customers	30	Registered buyer accounts with unique e-mail addresses
products	50	Catalog across 5 categories, priced ₹177.40 - ₹4,897.36
orders	400	Placed between 2024-06-15 and 2025-06-13 across 232 trading days
order_items	1,201	Line items - an average of 3 items per order
payments	400	One settled payment per order, across 4 methods
product_reviews	50	Star ratings from 1 to 5 with free-text review copy

What the data looks like

- **Categories.** Electronics (14 products), Clothing (13), Home (8), Toys (8), Books (7).
- **Order status.** Delivered 105, Shipped 102, Cancelled 99, Pending 94.
- **Payment methods.** Debit Card 113, Net Banking 98, Credit Card 96, UPI 93.
- **Trading window.** 2024-06-15 to 2025-06-13, covering 232 distinct order days.
- **Integrity.** Every order resolves to a customer and a payment; every product appears in at least one order.

CONSEQUENCE FOR TESTING

Because the dataset is complete, four queries legitimately return zero rows: orphaned orders (L2 Q1), out-of-stock products (L2 Q6), customers without orders (L5 Q4) and never-ordered products (L5 Q5). An empty result here is the correct answer, not a failed query.

LEVEL 1 · 10 QUESTIONS

Basics

Single-table retrieval

Level 1 establishes the reading vocabulary of SQL: choosing columns, removing duplicates, filtering rows and controlling order. Every query here touches exactly one table, so the focus is on stating the question precisely rather than on combining sources.

SELECT with named columns and SELECT * · DISTINCT · WHERE with comparison operators · BETWEEN, IN, LIKE · AND / OR / NOT · ORDER BY with multiple sort keys

L1 · Q1

Retrieve customer names and emails for email marketing

Lets the marketing team extract basic customer contact details for campaigns.

QUERY

```
SELECT name, email
FROM customers;
```

HOW IT WORKS

- SELECT chooses only the columns needed for the marketing list.
- FROM tells MySQL which table contains those columns.

EXPECTED RESULT 30 rows - one per customer, two columns (name, email).

name	email
Thomas Owens	user1@example.com
Charles Grant	user2@example.com
Kaitlin Richards	user3@example.com
Christina Williams	user4@example.com
David Allen	user5@example.com
Mark Duke	user6@example.com
Briana Wright	user7@example.com
John Bryan	user8@example.com
Jason Thompson	user9@example.com
Shawn Hill	user10@example.com
Walter Jenkins	user11@example.com
Mary Knight	user12@example.com
Leslie Wilson	user13@example.com
Deborah Arias	user14@example.com
Austin Flores	user15@example.com

L1 · Q2

View complete product catalog with all available details

The product manager can review every product listing in one go.

QUERY

```
SELECT *  
FROM products;
```

HOW IT WORKS

- * means "all columns" from the products table.
- This is useful when you want to inspect the complete table.

EXPECTED RESULT 50 rows × 6 columns - the full product catalog.

product_...	name	category	price	stock_quant...	added_on
1	Plant No	Home	639.43	152	2024-01-30 06:30:53
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
9	Common Study	Toys	985.19	171	2023-07-20 13:06:42
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
11	Build Her	Books	1852.64	150	2024-09-08 01:09:15
12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
13	Full West	Books	2112.33	172	2023-09-15 03:13:38
14	Listen Development	Home	296.17	132	2024-10-12 11:49:26
15	Place Low	Electronics	723.97	46	2023-07-05 14:36:07
16	Special Fact	Toys	1094.18	15	2025-03-17 10:42:40
17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55
18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53
19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01
20	Real Source	Books	4398.66	197	2025-02-08 10:28:27

L1 · Q3

List all unique product categories

Useful for analysing the range of departments and for building website filters.

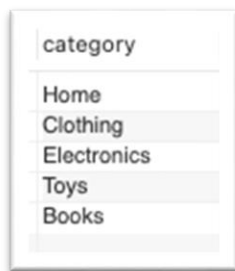
QUERY

```
SELECT DISTINCT category
FROM products;
```

HOW IT WORKS

- DISTINCT removes duplicate category values.
- Example: if Electronics appears 10 times, it appears once in the result.

EXPECTED RESULT 5 rows - Books, Clothing, Electronics, Home, Toys.



L1 · Q4

Show all products priced above ₹1,000

Identifies high-value items for premium promotions or pricing strategy reviews.

QUERY

```
SELECT *
FROM products
WHERE price > 1000;
```

HOW IT WORKS

- WHERE filters rows.
- Only products whose price is greater than 1000 are returned.

EXPECTED RESULT 40 of 50 products qualify.

product_...	name	category	price	stock_quant...	added_on
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
11	Build Her	Books	1852.64	150	2024-09-08 01:09:15
12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
13	Full West	Books	2112.33	172	2023-09-15 03:13:38
16	Special Fact	Toys	1094.18	15	2025-03-17 10:42:40
17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55
18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53
19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01
20	Real Source	Books	4398.66	197	2025-02-08 10:28:27
21	Red Including	Clothing	3845.31	92	2025-01-21 11:52:40

L1 · Q5

Display products within a mid-range price bracket (₹2,000 to ₹5,000)

A merchandising team needs this to create a mid-tier pricing campaign.

QUERY

```
SELECT *
FROM products
WHERE price BETWEEN 2000 AND 5000;
```

HOW IT WORKS

- BETWEEN checks whether a value falls within an inclusive range.
- Therefore 2000 and 5000 are both included.

EXPECTED RESULT 34 products fall inside the ₹2,000-₹5,000 bracket.

product_...	name	category	price	stock_quant...	added_on
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
13	Full West	Books	2112.33	172	2023-09-15 03:13:38
17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55
18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53
19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01
20	Real Source	Books	4398.66	197	2025-02-08 10:28:27
21	Bed Including	Clothing	3845.31	92	2025-01-21 11:52:40

L1 · Q6

Fetch data for specific customer IDs

Used when customer IDs are pre-selected in another system, such as a loyalty list.

QUERY

```
SELECT *  
FROM customers  
WHERE customer_id IN (1, 5, 10);
```

HOW IT WORKS

- IN lets us test a column against multiple values.
- Here we retrieve customers with IDs 1, 5, or 10.
- Replace these example IDs if your instructor specifies different IDs.

EXPECTED RESULT 3 rows - customers 1, 5 and 10.

customer_id	name	email	phone	created_at
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12
5	David Allen	user5@example.com	(751)456-8289x1	2023-10-29 02:43:00
10	Shawn Hill	user10@example.com	(268)113-3152x7	2023-12-14 20:46:43
NULL	NULL	NULL	NULL	NULL

L1 · Q7

Identify customers whose names start with the letter 'A'

Used for alphabetical segmentation in outreach or in-app directory display.

QUERY

```
SELECT *  
FROM customers  
WHERE name LIKE 'A%';
```

HOW IT WORKS

- LIKE is used for pattern matching.
- 'A%' means the name must start with A; % represents any remaining characters.

EXPECTED RESULT 4 customers - Austin Flores, Amy Landry, Amanda Bright, Angela Bailey.

customer_id	name	email	phone	created_at
15	Austin Flores	user15@example.com	329.901.1576x66	2024-06-13 09:03:42
16	Amy Landry	user16@example.com	+1-278-019-3748	2024-02-28 17:51:50
19	Amanda Bright	user19@example.com	380.981.9798x69	2024-12-20 22:58:15
27	Adrienne Green	user27@example.com	530.644.8455x93	2023-08-22 01:55:29
NULL	NULL	NULL	NULL	NULL

L1 · Q8

List electronics products priced under ₹3,000

Used by merchandising or frontend teams to showcase budget electronics.

QUERY

```
SELECT *  
FROM products  
WHERE category = 'Electronics'  
AND price < 3000;
```

HOW IT WORKS

- AND means both conditions must be true.
- The product must belong to Electronics AND cost less than 3000.

EXPECTED RESULT 7 Electronics products priced under ₹3,000.

product_...	name	category	price	stock_quant...	added_on
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
15	Place Low	Electronics	723.97	46	2023-07-05 14:36:07
31	Series Page	Electronics	2070.37	83	2024-04-01 00:24:06
34	Despite Win	Electronics	1340.34	64	2024-11-27 06:55:45
44	Actually Term	Electronics	396.11	85	2023-11-02 13:09:20
47	Southern Thing	Electronics	512.46	40	2024-02-28 17:57:38
NULL	NULL	NULL	NULL	NULL	NULL

L1 · Q9

Display product names and prices in descending order of price

Helps teams view and compare top-priced items quickly.

QUERY

```
SELECT name, price
FROM products
ORDER BY price DESC;
```

HOW IT WORKS

- ORDER BY sorts the result.
- DESC means highest values appear first.

EXPECTED RESULT 50 rows, highest price (₹4,897.36) first, lowest (₹177.40) last.

name	price
Response Indeed	4897.36
Population Social	4813.68
Development System	4801.78
Any Question	4759.28
Fire Often	4734.89
Natural Network	4722.66
Build High	4707.14
Serious Recognize	4523.10

L1 · Q10

Display product names and prices, sorted by price and then by name

Catalog teams list products most expensive first; ties break alphabetically for clean storefront and print output.

QUERY

```
SELECT name, price
FROM products
ORDER BY price DESC, name ASC;
```

HOW IT WORKS

- MySQL first sorts by price from high to low.
- When prices are equal, it sorts the names alphabetically (ASC).

EXPECTED RESULT 50 rows - identical to Q9, with alphabetical tie-breaking on equal prices.

name	price
Response Indeed	4897.36
Population Social	4813.68
Development System	4801.78
Any Question	4759.28
Fire Often	4734.89
Natural Network	4722.66
Build High	4707.14
Serious Recognize	4523.10
Study Total	4413.68
Real Source	4398.66

LEVEL 2 · 6 QUESTIONS

Filtering And Formatting

Shaping the output

Level 2 moves from retrieving data to presenting it. The result set is reshaped for humans and downstream systems - readable headings, derived columns, concatenated fields and dates stripped of their time component.

IS NULL / IS NOT NULL · Column and table aliases with AS · Arithmetic inside SELECT · CONCAT for string assembly · DATE() for timestamp formatting

L2 · Q1

Retrieve orders where customer information is missing

Identifies orphaned orders or test data where customer_id is not linked.

QUERY

```
SELECT *  
FROM orders  
WHERE customer_id IS NULL;
```

HOW IT WORKS

- IS NULL checks specifically for missing/NULL values.
- Do not use = NULL because NULL represents an unknown/missing value.

EXPECTED RESULT 1 row - Everything is NULL.

order_id	customer_id	order_date	status	total_amou...
NULL	NULL	NULL	NULL	NULL

L2 · Q2

Display customer names and emails using column aliases

Feeds frontend displays and report headings that need user-friendly labels.

QUERY

```
SELECT
  name AS customer_name,
  email AS customer_email
FROM customers;
```

HOW IT WORKS

- AS creates readable names for the output columns.
- Aliases change the displayed heading, not the underlying column.

EXPECTED RESULT 30 rows with headings customer_name and customer_email.

customer_name	customer_email
Thomas Owens	user1@example.com
Charles Grant	user2@example.com
Kaitlin Richards	user3@example.com
Christina Williams	user4@example.com
David Allen	user5@example.com
Mark Duke	user6@example.com
Briana Wright	user7@example.com
John Bryan	user8@example.com
Jason Thompson	user9@example.com
Shawn Hill	user10@example.com

L2 · Q3

Calculate total value per item ordered

Generates per-line item bill details and invoice breakdowns.

QUERY

```
SELECT
  order_item_id,
  order_id,
  product_id,
  quantity,
  item_price,
  quantity * item_price AS total_item_value
FROM order_items;
```

HOW IT WORKS

- Arithmetic can be performed directly inside SELECT.
- For each line item: quantity × item_price = total item value.

EXPECTED RESULT 1,201 rows - one per order line, with the derived total_item_value column.

order_item_...	order_id	product_...	quantity	item_price	total_item_val...
1	1	19	2	4707.14	9414.28
2	2	6	3	177.40	532.20
3	3	9	3	985.19	2955.57
4	3	23	1	2208.99	2208.99
5	4	35	2	4734.89	9469.78
6	5	19	1	4707.14	4707.14
7	5	7	2	4897.36	9794.72
8	6	7	3	4897.36	14692.08
9	6	37	1	1429.45	1429.45
10	6	15	1	723.97	723.97
11	6	35	3	4734.89	14204.67
12	7	30	1	3043.67	3043.67
13	8	10	3	4801.78	14405.34
14	8	5	3	4722.66	14167.98
15	8	31	2	2070.37	4140.74
16	8	7	2	4897.36	14692.08

L2 · Q4

Combine customer name and phone number in a single column

Produces brief customer summaries or contact lists.

QUERY

```
SELECT
  CONCAT(name, ' - ', phone) AS customer_contact
FROM customers;
```

HOW IT WORKS

- CONCAT joins multiple text values into one string.
- The AS alias gives the calculated column a readable heading.

EXPECTED RESULT 30 rows in the format 'Name - Phone'.

customer_contact
Thomas Owens - 142-479-1945
Charles Grant - 9153947511
Kaitlin Richards - 2073473421
Christina Williams - 586-605-5061x06
David Allen - (751)456-8289x1
Mark Duke - (144)957-2811
Briana Wright - 223-833-9635
John Bryan - 045.568.0798x27
Jason Thompson - 1862659420
Shawn Hill - (268)113-3152x7
Walter Jenkins - 536-329-0817x71
Mary Knight - 361-636-3802
Leslie Wilson - +1-256-261-1984

L2 · Q5

Extract only the date part from order timestamps

Allows orders to be grouped or filtered by date without the time component.

QUERY

```
SELECT
  order_id,
  DATE(order_date) AS order_date
FROM orders;
```

HOW IT WORKS

- DATE() removes the time portion and returns only YYYY-MM-DD.

EXPECTED RESULT 400 rows - order_id with the timestamp reduced to YYYY-MM-DD.

order_id	order_date
1	2025-03-02
2	2024-10-09
3	2025-05-08
4	2024-09-19
5	2025-04-08
6	2024-10-25
7	2024-07-29
8	2024-07-30
9	2025-06-10

L2 · Q6

List products that do not have any stock left

Helps the inventory team identify out-of-stock items.

QUERY

```
SELECT *  
FROM products  
WHERE stock_quantity = 0;
```

HOW IT WORKS

- The WHERE condition keeps only products whose inventory is exactly zero.

EXPECTED RESULT 1 row – Everything is NULL.

product_...	name	category	price	stock_quant...	added_on
NULL	NULL	NULL	NULL	NULL	NULL

LEVEL 3 · 10 QUESTIONS

Aggregations

Summarising many rows into one

Level 3 collapses detail into insight. Aggregate functions reduce a set of rows to a single value, and GROUP BY splits the table into buckets so that the summary is produced per customer, per category, per day or per payment method.

COUNT, SUM, AVG, MIN, MAX · GROUP BY on one or more columns · ROUND for presentation-grade numbers · Aggregates combined with joins

L3 · Q1

Count the total number of orders placed

Used by business managers to track order volume over time.

QUERY

```
SELECT COUNT(*) AS total_orders
FROM orders;
```

HOW IT WORKS

- COUNT(*) counts every row in the orders table.
- AS total_orders gives the result a meaningful column name.

EXPECTED RESULT 1 row - 400 orders.

total...
400

L3 · Q2

Calculate the total revenue collected from all orders

Gives the overall sales value of the business.

QUERY

```
SELECT SUM(total_amount) AS total_revenue
FROM orders;
```

HOW IT WORKS

- SUM() adds all values in total_amount.

EXPECTED RESULT 1 row - ₹69,60,973.66 total revenue.

total_revenue
6960973.66

L3 · Q3

Calculate the average order value

Used to understand customer spending patterns.

QUERY

```
SELECT ROUND(AVG(total_amount), 2) AS average_order_value
FROM orders;
```

HOW IT WORKS

- AVG() calculates the mean order amount.
- ROUND(AVG(total_amount), 2) keeps the result to two decimal places.

EXPECTED RESULT 1 row - ₹17,402.43 average order value.

average_order_val...
17402.43

L3 · Q4

Count the number of customers who have placed at least one order

Identifies the active customer base.

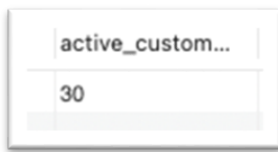
QUERY

```
SELECT COUNT(DISTINCT customer_id) AS active_customers
FROM orders
WHERE customer_id IS NOT NULL;
```

HOW IT WORKS

- DISTINCT prevents the same customer from being counted multiple times.
- IS NOT NULL excludes orders that have no customer ID.

EXPECTED RESULT 1 row - 30 active customers (100% of the customer base).



active_custom...
30

L3 · Q5

Find the number of orders placed by each customer

Helpful for identifying top or repeat customers.

QUERY

```
SELECT
  customer_id,
  COUNT(*) AS order_count
FROM orders
GROUP BY customer_id;
```

HOW IT WORKS

- GROUP BY creates one group for each customer.
- COUNT(*) then counts the orders inside each group.

EXPECTED RESULT 30 rows; order counts range from 8 to 20 per customer.

customer_id	order_count
1	12
2	17
3	17
4	9
5	14
6	16
7	13
8	10
9	10
10	13
11	9
12	11

L3 · Q6

Find total sales amount made by each customer

Ranks customers by lifetime value for account and loyalty decisions.

QUERY

```
SELECT
  customer_id,
  SUM(total_amount) AS total_sales
FROM orders
GROUP BY customer_id;
```

HOW IT WORKS

- GROUP BY separates orders by customer.
- SUM(total_amount) adds the order values within each customer group.

EXPECTED RESULT 30 rows of customer lifetime revenue.

customer_id	total_sales
1	183747.44
2	284420.07
3	253783.31
4	137562.22
5	262504.19
6	212173.58
7	167960.11
8	164701.78
9	106226.03
10	252722.75
11	173409.01

L3 · Q7

List the number of products sold per category

Helps category managers assess performance by department.

QUERY

```
SELECT
  p.category,
  SUM(oi.quantity) AS products_sold
FROM products AS p
INNER JOIN order_items AS oi
  ON p.product_id = oi.product_id
GROUP BY p.category;
```

HOW IT WORKS

- JOIN connects products to the order_items table using product_id.
- SUM(quantity) gives the number of units sold.
- GROUP BY category produces one result per product category.

EXPECTED RESULT 5 rows - Electronics 687 units, Clothing 559, Home 443, Toys 405, Books 350.

category	products_s...
Home	443
Clothing	559
Electronics	687
Toys	405
Books	350

L3 · Q8

Find the average item price per category

Useful for comparing realised pricing across departments.

QUERY

```
SELECT
  p.category,
  ROUND(AVG(oi.item_price), 2) AS average_item_price
FROM products AS p
INNER JOIN order_items AS oi
  ON p.product_id = oi.product_id
GROUP BY p.category;
```

HOW IT WORKS

- AVG() calculates the average item price within each category.
- The JOIN connects each order item to its product category.

EXPECTED RESULT 5 rows - Clothing ₹3,473.17, Books ₹3,216.39, Electronics ₹2,733.93, Toys ₹2,407.63, Home ₹2,330.71.

category	average_item_pri...
Home	2330.71
Clothing	3473.17
Electronics	2733.93
Toys	2407.63
Books	3216.39

L3 · Q9

Show number of orders placed per day

Used to track daily business activity and demand trends.

QUERY

```
SELECT
  DATE(order_date) AS order_day,
  COUNT(*) AS order_count
FROM orders
GROUP BY DATE(order_date)
ORDER BY order_day;
```

HOW IT WORKS

- DATE(order_date) groups orders by calendar date instead of timestamp.
- COUNT(*) counts how many orders occurred on each day.

EXPECTED RESULT 232 rows - one per trading day between 2024-06-15 and 2025-06-13.

order_day	order_count
2024-06-15	1
2024-06-18	1
2024-06-19	3
2024-06-20	2
2024-06-21	2
2024-06-23	2
2024-06-24	1
2024-06-25	1

L3 · Q10

List total payments received per payment method

Helps the finance team understand preferred transaction modes.

QUERY

```
SELECT
  method,
  SUM(amount_paid) AS total_payment
FROM payments
GROUP BY method;
```

HOW IT WORKS

- GROUP BY creates one group for each payment method.
- SUM(amount_paid) calculates the total received through each method.

EXPECTED RESULT 4 rows - Debit Card ₹19,30,577.88, Credit Card ₹17,54,603.10, Net Banking ₹16,58,383.90, UPI ₹16,17,408.78.

method	total_payme...
Credit Card	1754603.10
Net Banking	1658383.90
UPI	1617408.78
Debit Card	1930577.88

LEVEL 4 · 7 QUESTIONS

Multi-Table Queries (Joins)

Connecting the schema

Level 4 reunites data that normalisation deliberately separated. The join type is the decision that matters: INNER JOIN answers 'what matched', LEFT and RIGHT JOIN answer 'what matched and what did not'.

INNER JOIN · LEFT JOIN · RIGHT JOIN · Table aliases · Chaining multiple joins in one query

L4 · Q1

Retrieve order details along with the customer name (INNER JOIN)

Used for displaying which customer placed each order.

QUERY

```
SELECT
  o.order_id,
  c.name AS customer_name,
  o.order_date,
  o.status,
  o.total_amount
FROM orders AS o
INNER JOIN customers AS c
  ON o.customer_id = c.customer_id;
```

HOW IT WORKS

- INNER JOIN returns rows where a matching customer exists for the order.
- o and c are table aliases used to keep the query shorter.

EXPECTED RESULT 400 rows - every order matches a valid customer.

order_id	customer_name	order_date	status	total_amou...
14	Thomas Owens	2024-09-24 21:21:38	Shipped	15803.34
17	Thomas Owens	2024-08-19 21:17:57	Pending	11173.77
61	Thomas Owens	2024-12-26 23:21:58	Shipped	13053.00
76	Thomas Owens	2025-01-14 22:59:54	Pending	23506.81
92	Thomas Owens	2024-09-26 11:33:58	Shipped	834.75
109	Thomas Owens	2025-03-17 04:56:18	Shipped	12190.14
127	Thomas Owens	2025-06-10 18:01:32	Pending	20160.64
135	Thomas Owens	2025-03-11 22:58:23	Pending	8891.79
144	Thomas Owens	2024-09-19 09:18:22	Shipped	12093.54
221	Thomas Owens	2024-09-25 05:49:39	Pending	12437.61
278	Thomas Owens	2025-02-01 20:05:38	Deliver...	32015.16
379	Thomas Owens	2025-03-22 15:57:58	Cancel...	21586.89
56	Charles Grant	2024-06-23 15:59:57	Pending	6828.61
169	Charles Grant	2025-02-11 04:04:38	Cancel...	9426.30
209	Charles Grant	2024-10-16 10:07:48	Shipped	7690.62

L4 · Q2

Get list of products that have been sold (INNER JOIN with order_items)

Used to find which products were actually included in orders.

QUERY

```
SELECT DISTINCT
  p.product_id,
  p.name,
  p.category,
  p.price
FROM products AS p
INNER JOIN order_items AS oi
  ON p.product_id = oi.product_id;
```

HOW IT WORKS

- A product is considered sold when it has a matching row in order_items.
- DISTINCT prevents a product from appearing multiple times if it was sold repeatedly.

EXPECTED RESULT 50 rows - all 50 products have been sold at least once.

product_...	name	category	price	
1	Plant No	Home	639.43	
2	Population Social	Clothing	4813.68	
3	Available Answer	Electronics	2529.51	
4	Any Question	Clothing	4759.28	
5	Natural Network	Toys	4722.66	
6	If Whatever	Electronics	177.40	
7	Response Indeed	Clothing	4897.36	
8	Every Amount	Home	4173.60	

L4 · Q3

List all orders with their payment method (INNER JOIN)

Used by finance or audit teams to see how each order was paid for.

QUERY

```

SELECT
  o.order_id,
  o.order_date,
  o.total_amount,
  p.method
FROM orders AS o
INNER JOIN payments AS p
  ON o.order_id = p.order_id;

```

HOW IT WORKS

- The order_id column is the common key between orders and payments.
- INNER JOIN returns orders that have a matching payment record.

EXPECTED RESULT 400 rows - every order carries exactly one payment record.

order_id	order_date	total_amou...	method	
1	2025-03-02 07:20:11	9414.28	Credit Card	
2	2024-10-09 18:08:21	532.20	Net Banking	
3	2025-05-08 00:08:27	5164.56	Credit Card	
4	2024-09-19 22:16:13	9469.78	UPI	
5	2025-04-08 18:02:06	14501.86	UPI	
6	2024-10-25 07:33:59	31050.17	UPI	
7	2024-07-29 11:58:47	3043.67	UPI	
8	2024-07-29 00:10:10	2274.22	Net Banking	

L4 · Q4

Get list of customers and their orders (LEFT JOIN)

Used to find all customers and see who has or hasn't placed orders.

QUERY

```
SELECT
  c.customer_id,
  c.name,
  o.order_id,
  o.order_date,
  o.total_amount
FROM customers AS c
LEFT JOIN orders AS o
  ON c.customer_id = o.customer_id;
```

HOW IT WORKS

- LEFT JOIN keeps every customer, even if they have no order.
- Customers without orders will have NULL values in the order columns.

EXPECTED RESULT 400 rows - no NULL order columns, so no customer is order-less.

	customer_id	name	order_id	order_date	total_amou...	
	1	Thomas Owens	14	2024-09-24 21:21:38	15803.34	
	1	Thomas Owens	17	2024-08-19 21:17:57	11173.77	
	1	Thomas Owens	61	2024-12-26 23:21:58	13053.00	
	1	Thomas Owens	76	2025-01-14 22:59:54	23506.81	
	1	Thomas Owens	92	2024-09-26 11:33:58	834.75	
	1	Thomas Owens	109	2025-03-17 04:56:18	12190.14	
	1	Thomas Owens	127	2025-06-10 18:01:32	20160.64	
	1	Thomas Owens	135	2025-03-11 22:58:23	8891.79	
	1	Thomas Owens	144	2024-09-19 09:18:22	12093.54	
	1	Thomas Owens	221	2024-09-25 05:49:39	12437.61	
	1	Thomas Owens	278	2025-02-01 20:05:38	32015.16	

L4 · Q5

List all products along with order item quantity (LEFT JOIN)

Useful for inventory teams tracking what sold and what didn't.

QUERY

```
SELECT
  p.product_id,
  p.name,
  p.category,
  oi.order_id,
  oi.quantity
FROM products AS p
LEFT JOIN order_items AS oi
  ON p.product_id = oi.product_id;
```

HOW IT WORKS

- LEFT JOIN keeps every product, including products that were never ordered.
- For never-ordered products, order_id and quantity will be NULL.

EXPECTED RESULT 1,201 rows - one per order line; no product returns NULL.

product_...	name	category	order_id	quantity
1	Plant No	Home	10	1
1	Plant No	Home	11	3
1	Plant No	Home	28	1
1	Plant No	Home	37	1
1	Plant No	Home	49	2
1	Plant No	Home	71	3
1	Plant No	Home	75	2
1	Plant No	Home	81	2
1	Plant No	Home	127	2
1	Plant No	Home	134	2

L4 · Q6

List all payments including those with no matching orders (RIGHT JOIN)

Rare, but used when ensuring every payment is mapped to a valid order.

QUERY

```
SELECT
  o.order_id,
  o.order_date,
  p.payment_id,
  p.payment_date,
  p.amount_paid,
  p.method
FROM orders AS o
RIGHT JOIN payments AS p
  ON o.order_id = p.order_id;
```

HOW IT WORKS

- RIGHT JOIN keeps every row from the payments table.
- If a payment has no matching order, order columns become NULL.

EXPECTED RESULT 400 rows - no unmatched payments, so no NULL order columns appear.

	order_id	order_date	payment_id	payment_date	amount_paid	method	
	1	2025-03-02 07:20:11	1	2025-03-02 08:16:11	9414.28	Credit Card	
	2	2024-10-09 18:08:21	2	2024-10-09 19:00:21	532.20	Net Banking	
	3	2025-05-08 00:08:27	3	2025-05-08 00:55:27	5164.56	Credit Card	
	4	2024-09-19 22:16:13	4	2024-09-19 22:28:13	9469.78	UPI	
	5	2025-04-08 18:02:06	5	2025-04-08 18:26:06	14501.86	UPI	
	6	2024-10-25 07:33:59	6	2024-10-25 08:16:59	31050.17	UPI	
	7	2024-07-29 11:58:47	7	2024-07-29 12:45:47	3043.67	UPI	

L4 · Q7

Combine data from three tables: customer, order, and payment

Used for detailed end-to-end transaction reports.

QUERY

```
SELECT
  c.customer_id,
  c.name AS customer_name,
  o.order_id,
  o.order_date,
  o.total_amount,
  p.payment_id,
  p.amount_paid,
  p.method
FROM customers AS c
INNER JOIN orders AS o
  ON c.customer_id = o.customer_id
INNER JOIN payments AS p
  ON o.order_id = p.order_id;
```

HOW IT WORKS

- This uses two INNER JOIN operations.
- First: customers → orders using customer_id.
- Second: orders → payments using order_id.

EXPECTED RESULT 400 rows - the complete customer → order → payment chain.

customer_id	customer_name	order_id	order_date	total_amou...	payment_id	amount_paid	method
1	Thomas Owens	14	2024-09-24 21:21:38	15803.34	14	15803.34	UPI
1	Thomas Owens	17	2024-08-19 21:17:57	11173.77	17	11173.77	UPI
1	Thomas Owens	61	2024-12-26 23:21:58	13053.00	61	13053.00	Net Banking
1	Thomas Owens	76	2025-01-14 22:59:54	23506.81	76	23506.81	Credit Card
1	Thomas Owens	92	2024-09-26 11:33:58	834.75	92	834.75	Net Banking
1	Thomas Owens	109	2025-03-17 04:56:18	12190.14	109	12190.14	UPI
1	Thomas Owens	127	2025-06-10 18:01:32	20160.64	127	20160.64	Debit Card

LEVEL 5 · 7 QUESTIONS

Subqueries (Inner Queries)

Queries inside queries

Level 5 introduces questions whose filter depends on a value the database must calculate first. Scalar subqueries supply a single comparison value; correlated subqueries recalculate per row; EXISTS and NOT EXISTS test for presence and absence.

Scalar subquery in WHERE · Correlated subqueries · EXISTS / NOT EXISTS · Aggregates inside subqueries

L5 · Q1

List all products priced above the average product price

Used by pricing analysts to identify premium-priced products.

QUERY

```
SELECT *  
FROM products  
WHERE price > (  
    SELECT AVG(price)  
    FROM products  
);
```

HOW IT WORKS

- The inner query calculates the average price.
- The outer query returns products whose price is greater than that average.

EXPECTED RESULT 27 products priced above the ₹2,825.39 catalog average.

product_...	name	category	price	stock_quant...	added_on
2	Populat...	Clothing	4813.68	84	2025-05-30 10:02:50
4	Any Qu...	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural...	Toys	4722.66	75	2023-11-06 00:47:37
7	Respon...	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every A...	Home	4173.60	156	2025-04-30 03:11:10
10	Develo...	Electronics	4801.78	153	2025-03-12 08:22:57
12	Action...	Electronics	4017.01	19	2025-02-14 03:38:06
18	Some...	Toys	3673.86	110	2024-07-25 00:33:53

L5 · Q2

Find customers who have placed at least one order

Used to identify active customers for loyalty campaigns.

QUERY

```
SELECT *  
FROM customers AS c  
WHERE EXISTS (  
  SELECT 1  
  FROM orders AS o  
  WHERE o.customer_id = c.customer_id  
);
```

HOW IT WORKS

- EXISTS checks whether at least one matching order exists for the customer.
- SELECT 1 is enough because EXISTS only cares whether a row exists.

EXPECTED RESULT 30 rows - every customer has ordered at least once.

	customer_id	name	email	phone	created_at	
1		Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	
2		Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24	
3		Kaitlin Richards	user3@example.com	2073473421	2024-06-23 09:55:22	
4		Christina Williams	user4@example.com	586-605-5061x06	2024-10-27 17:19:38	
5		David Allen	user5@example.com	(751)456-8289x1	2023-10-29 02:43:00	
6		Mark Duke	user6@example.com	(144)957-2811	2024-06-24 03:22:59	
7		Briana Wright	user7@example.com	223-833-9635	2023-06-25 00:35:43	

L5 · Q3

Show orders whose total amount is above the average for that customer

Used to detect unusually high purchases relative to a customer's own norm.

QUERY

```
SELECT
  o.order_id,
  o.customer_id,
  o.total_amount
FROM orders AS o
WHERE o.total_amount > (
  SELECT AVG(o2.total_amount)
  FROM orders AS o2
  WHERE o2.customer_id = o.customer_id
);
```

HOW IT WORKS

- This is a correlated subquery.
- For each order, the inner query calculates that customer's average order value.
- The outer query keeps the order only when it is above that average.

EXPECTED RESULT 194 of 400 orders exceed their own customer's average.

order_id	customer_id	total_amou...
6	29	31050.17
8	19	32714.06
9	6	24219.20
10	28	24342.52
14	1	15803.34
16	5	22856.73
18	13	32001.24

L5 · Q4

Display customers who haven't placed any orders

Used for re-engagement campaigns targeting inactive users.

QUERY

```
SELECT *
FROM customers AS c
WHERE NOT EXISTS (
  SELECT 1
  FROM orders AS o
  WHERE o.customer_id = c.customer_id
);
```

HOW IT WORKS

- NOT EXISTS is the opposite of EXISTS.
- It returns customers for whom no matching order can be found.

EXPECTED RESULT 1 row – NULL, there are no dormant customers in this dataset.

customer_id	name	email	phone	created_at
NULL	NULL	NULL	NULL	NULL

L5 · Q5

Show products that were never ordered

Helps with inventory clearance decisions or product deactivation.

QUERY

```
SELECT *
FROM products AS p
WHERE NOT EXISTS (
  SELECT 1
  FROM order_items AS oi
  WHERE oi.product_id = p.product_id
);
```

HOW IT WORKS

- The subquery looks for a matching order_items row.
- NOT EXISTS keeps products for which no such row exists.

EXPECTED RESULT 1 row – NULL, every product has been ordered at least once.

product_...	name	category	price	stock_quant...	added_on
NULL	NULL	NULL	NULL	NULL	NULL

L5 • Q6

Show highest value order per customer

Used to identify the largest transaction made by each customer.

QUERY

```
SELECT
  customer_id,
  MAX(total_amount) AS highest_order_value
FROM orders
GROUP BY customer_id;
```

HOW IT WORKS

- GROUP BY creates one group per customer.
- MAX() finds the largest order amount inside each group.

EXPECTED RESULT 30 rows - the peak order value for each customer.

customer_id	highest_order_val...
1	32015.16
2	42056.04
3	41679.11
4	25747.34
5	39921.78
6	39003.19
7	28589.04

L5 · Q7

Highest Order Per Customer (Including Names)

Same as above, but returns the customer name and order for reporting.

QUERY

```
SELECT
  c.customer_id,
  c.name,
  o.order_id,
  o.total_amount
FROM customers AS c
INNER JOIN orders AS o
  ON c.customer_id = o.customer_id
WHERE o.total_amount = (
  SELECT MAX(o2.total_amount)
  FROM orders AS o2
  WHERE o2.customer_id = o.customer_id
);
```

HOW IT WORKS

- The inner query finds the maximum order amount for the current customer.
- The outer query returns the actual order and customer name matching that value.

EXPECTED RESULT 30 rows - peak order value with customer name and order_id attached.

	customer_id	name	order_id	total_amou...
1		Thomas Owens	278	32015.16
2		Charles Grant	315	42056.04
3		Kaitlin Richards	281	41679.11
4		Christina Williams	386	25747.34
5		David Allen	266	39921.78
6		Mark Duke	330	39003.19
7		Briana Wright	128	28589.04

LEVEL 6 · 2 QUESTIONS

Set Operations

Combining result sets

Level 6 treats query results as mathematical sets. UNION merges two result sets and removes duplicates. INTERSECT is unavailable in the target MySQL version, so the intersection is expressed with an INNER JOIN instead.

UNION · Deduplication semantics · INNER JOIN as an INTERSECT substitute

L6 · Q1

List all customers who have either placed an order or written a product review

Used to identify engaged customers across different activity areas.

QUERY

```
SELECT customer_id
FROM orders
WHERE customer_id IS NOT NULL

UNION

SELECT customer_id
FROM product_reviews
WHERE customer_id IS NOT NULL;
```

HOW IT WORKS

- UNION combines the two result sets.
- UNION removes duplicate customer IDs automatically.

EXPECTED RESULT 30 distinct customer IDs - the full engaged base.

customer_id
1
2
3
4
5
6
7

L6 · Q2

List all customers who have placed an order as well as reviewed a product.

Used to identify highly engaged customers for rewards.

QUERY

```
SELECT DISTINCT
  o.customer_id
FROM orders AS o
INNER JOIN product_reviews AS pr
  ON o.customer_id = pr.customer_id;
```

HOW IT WORKS

- The PDF notes that INTERSECT is not supported in the intended MySQL setup.
- INNER JOIN produces the intersection: customers appearing in both tables.
- DISTINCT ensures each customer ID appears only once.

EXPECTED RESULT 23 customer IDs appear in both orders and product_reviews.

customer_id
1
2
4
6
7
9
10
11

12 - OUTCOME

Key Findings

What the 42 queries actually reveal about the business.

- **Revenue.** ₹69,60,973.66 across 400 orders, an average order value of ₹17,402.43.
- **Engagement is total.** All 30 customers have ordered and all 50 products have sold - there is no dormant segment and no dead stock in this dataset.
- **Category volume.** Electronics leads on units sold (687), but Clothing commands the highest average realised item price (₹3,473.17).

- **Payments.** Debit Card settles the largest value (₹19,30,577.88); UPI the smallest (₹16,17,408.78). No single method dominates.
- **Order behaviour.** 194 of 400 orders - roughly 49% - exceed the placing customer's own average order value, indicating frequent high-value outliers rather than uniform spending.
- **Review coverage.** Only 23 of 30 customers have both ordered and reviewed, making review participation the clearest lever for engagement campaigns.

13 - NOTES

Assumptions & Engine Notes

Decisions were made in cases where the brief allowed for interpretation.

- **INTERSECT is unsupported.** The brief flags this for Level 6 Q2. The intersection is produced with an `INNER JOIN` plus `DISTINCT`, which returns the same set.
- **Loyalty IDs are illustrative.** Level 1 Q6 filters on customer IDs 1, 5 and 10; substitute the IDs supplied by the instructor if they differ.
- **NULL comparison.** `IS NULL` is used rather than `= NULL`, because `NULL` denotes an unknown value and never equals anything, including itself.
- **RIGHT JOIN retained.** Level 4 Q6 keeps `RIGHT JOIN` as the brief requests, even though the equivalent `LEFT JOIN` with the tables reversed is more conventional.
- **Currency.** Amounts are treated as Indian Rupees, matching the ₹ notation in the brief.
- **Rounding.** `ROUND(..., 2)` is applied to averages for presentation only; the underlying values are not modified.

14 - EXECUTION

How To Run

Reproducing the results in this document on a clean MySQL server.

1. Open MySQL Workbench and connect to the local server instance.
2. Open `Awanish Verma - Retail_Store_Complete_Solution_Explained.sql`.
3. Execute the script from the top through the verification query. Confirm the counts read 30, 50, 400, 1201, 400, and 50.
4. Run each level's queries individually and capture the result grid for submission.

5. If the script is re-run, `INSERT IGNORE` and `IF NOT EXISTS` make it safe; no duplicate-key errors will occur.